

AI n U · [Follow publication](#)

Convolutional implementation of the sliding window algorithm

Dig dive into the Object Detection notion

7 min read · Jan 29, 2020



Rrohan.Arrora

Follow



Listen



Share

In this article, I will discuss how the sliding window algorithm can be understood using the convolutional algorithm. But before that, let us build the intuition for it.

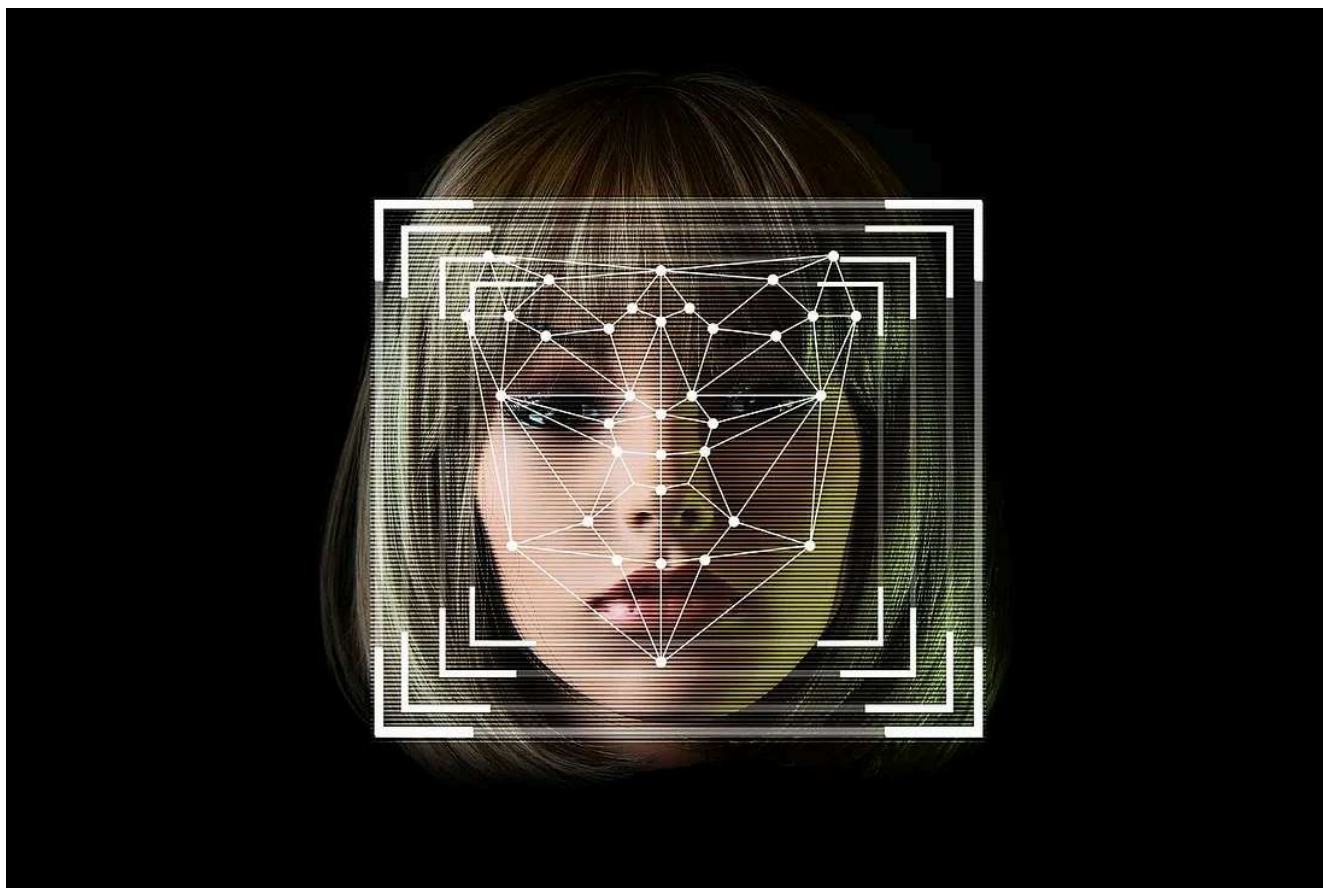


Image by [Gerd Altmann](#) from [Pixabay](#)

— Object classification and localization

Object detection includes the object classification and the object localization concepts in great detail. So, it becomes more evident to understand these concepts. Image classification is a more general concept of classifying the type of image from a bunch of classes. Image classification includes various strategies like traditional neural network or convolutional neural network, etc.

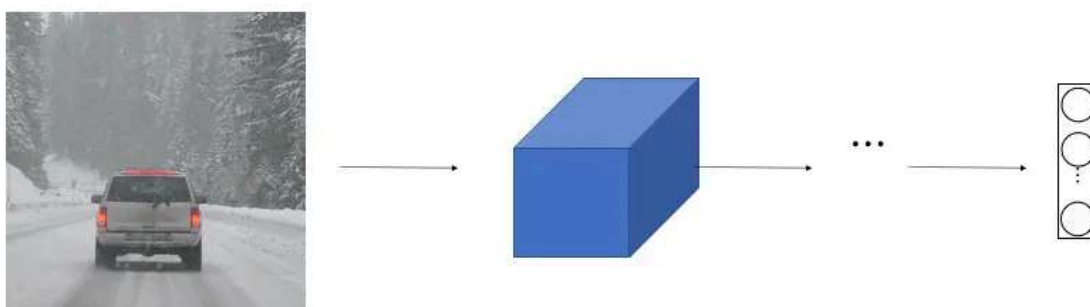
Image localization is finding the boundaries of the object in the image. You can easily see in the below image a box around the car that states the position of the object in the image.



Image by [Isa KARAKUS](#) from [Pixabay](#).

Now, object detection is simply detecting multiple objects in the image using two above concepts i.e. classification and localization.

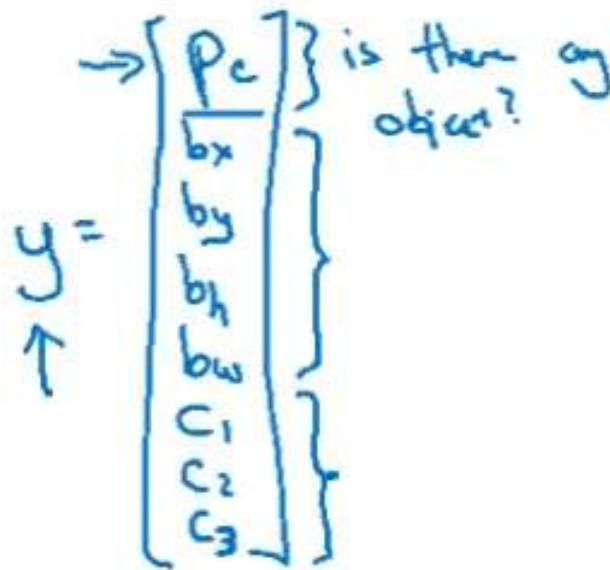
— Defining the target label y



The output y will include the values for the below sections:

1. If the image contains an object or not(pc).
2. The bounding box coordinates including the center of the image, width of the image and the height of the image(bx, by, bh, bw).

3. Class of the object(c1, c2, c3).



Defining variable y

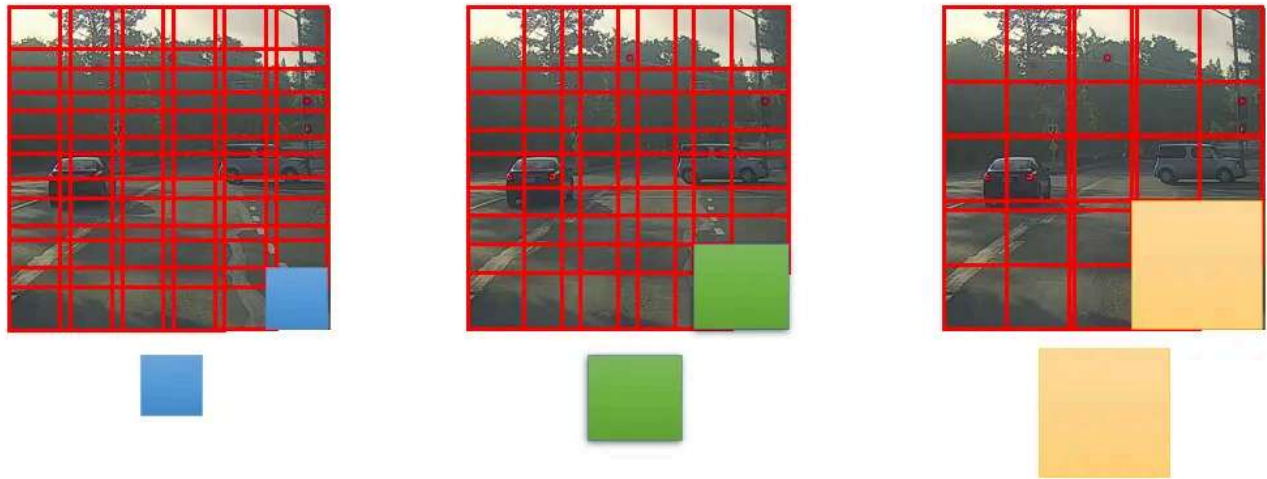
Object localization is also known as landmark detection where we try to detect the presence of the object in the image. Now, we could extract different types of information from the landmark detection like the pose of the person in the image, type of smile of the person, etc.

— The sliding windows detection algorithm

In object detection problems, we generally have to find all the possible objects in the image like all the cars in the image, all the pedestrians in the image, all the bikes in the image, etc. To achieve this, we use an algorithm known as **Sliding window detection**. Let us understand this algorithm.

- In this algorithm, we choose a grid cell of a specific size. Let us choose the grid cell of size 2x2.
- We pass the above grid cell through the image and convolute the part of the image in the grid cell and predict the output.

- Then we slide the grid cell through stride-2 and then convolute the next part of the image.
- In this way, we go cover the whole image.
- We repeat the same procedure with different grid cells size.

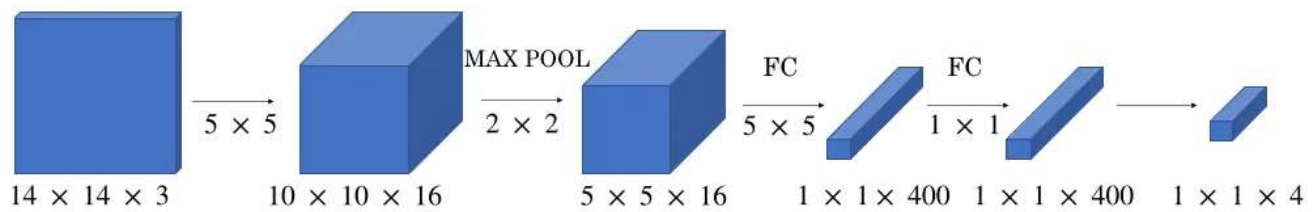


Disadvantages of the sliding window:

- It is computationally expensive (Earlier simple linear classifiers were used which were not as expensive to compute as compared to neural networks.).
- Time-consuming.

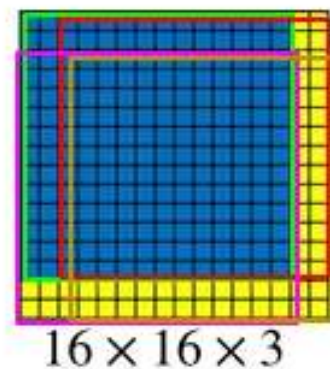
Fortunately, however, this problem of computational cost has a pretty good solution. It can be handled by the *convolutional implementation of the sliding window algorithm*.

We train our development dataset on the below convolutional neural network.



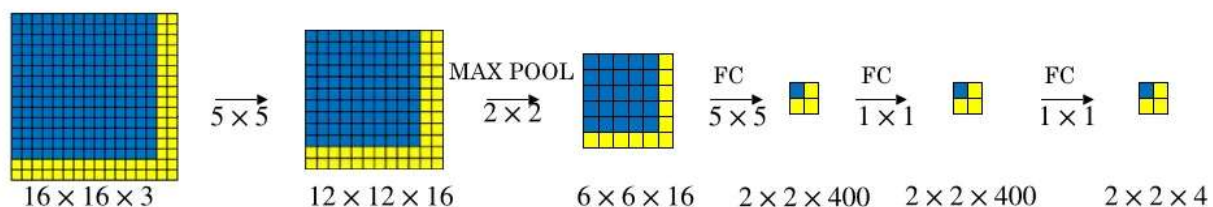
But in our test dataset, we get the image of size $16 \times 16 \times 3$. So, there are two ways:

1. Sliding window approach:



We pass through $14 \times 14 \times 3$ image size through the above convolutional neural network and then try to predict the class of the object in the image.

2. Convolutional approach:



We apply convolutional techniques to the whole image and output a $2 \times 2 \times 4$ image. Each $1 \times 1 \times 4$ in the output corresponds to one of the sliding windows as shown in different colors above. Moreover, this

convolutional replacement of the sliding window protocol is much more economical.

Get Rrohan.Arrora's stories in your inbox

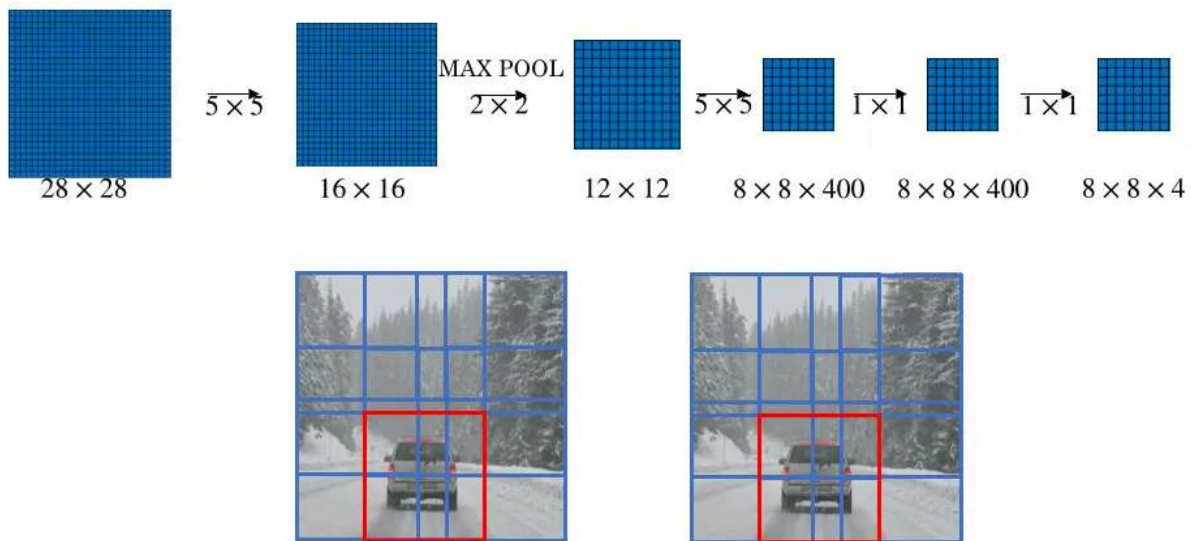
Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Let us understand the above concept for the 28x28 image.



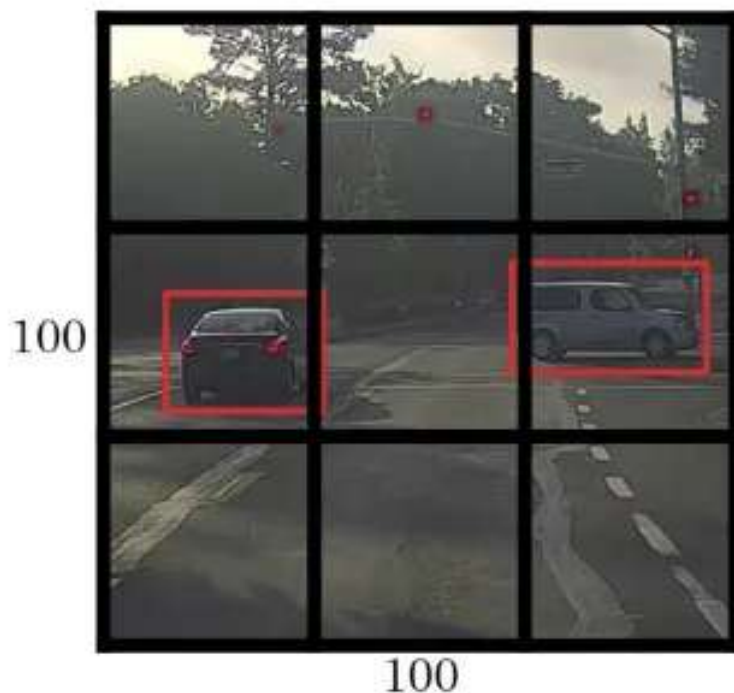
Andrew Ng

Each 1x1x4 block in the output image corresponding to one of the sliding windows in the image as shown above. Now, this algorithm still has one weakness, the position of the bounding boxes is not too accurate.

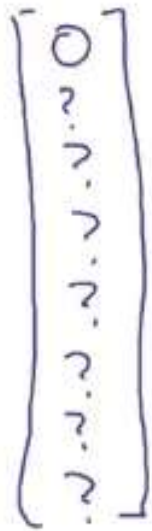
This is how the sliding window algorithm is implemented convolutionally.

— You Only Look Once(YOLO)

“You Only Look Once” (YOLO) is a popular algorithm because it achieves high accuracy while also being able to run in real-time. This algorithm “**only looks once**” at the image in the sense that it requires only one forward propagation pass through the network to make predictions. After non-max suppression, it then outputs recognized objects together with the bounding boxes.

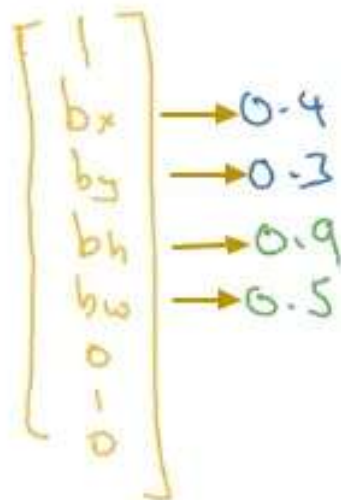


- To understand the above algorithm, we take a 100x100 image and place 3x3 grid cells over it.
- We take image classification and localization algorithm & apply that to each of the nine grid cells of this image.
- Now, our labels would include pc, bx, by, bh, bw, c1, c2, c3. Label for upper left grid cell would be



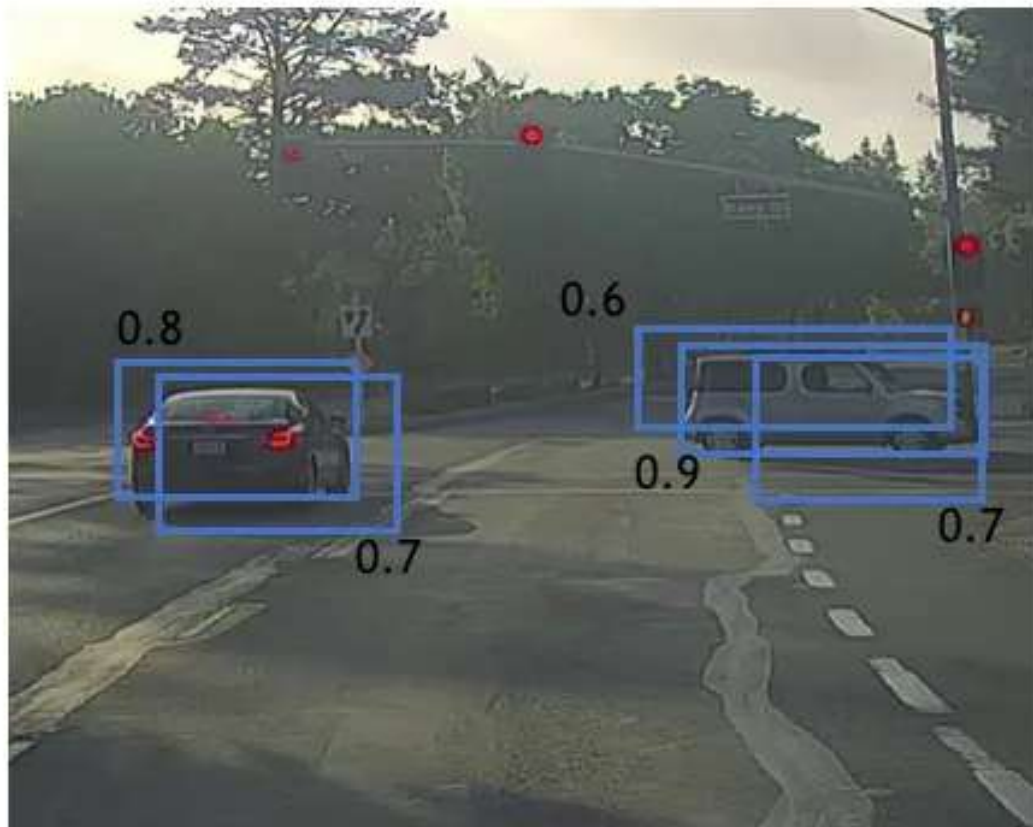
? represents don't care.

- As per the Yolo algorithm, the object is assigned to the grid cell whose midpoint lies in it. Therefore, the first car object is assigned to the 4th grid cell and the second car object is assigned to the 6th grid cell.
- The target output in the above case would be $3 \times 3 \times 8$ in correspondence to each grid cell.
- The bounding box for the second car would be:



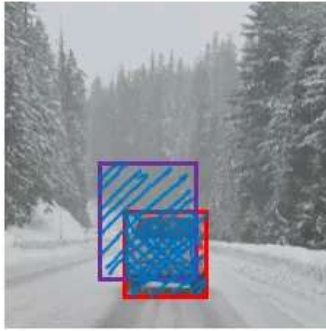
b_x and b_y would be smaller than 1 as they represent the grid cell where the midpoint of the object lies whereas b_h and b_w could be larger than 1 as the image could be present in more than 1 grid cell.

— Intersection over union & Non-max suppression



In some cases, we get more than 1 bounding box for the object detection. In order to handle this, we apply the non-max suppression algorithm. Before understanding the algorithm, let us understand IoU first.

As per the IoU, we can choose the most accurate bounding box among the other bounding boxes for the same object using IoU algorithm. More generally, IoU is a measure of the overlap between two bounding boxes.



$$\text{Intersection over Union (IoU)} = \frac{\text{Size of } \boxed{\text{Intersection}}}{\text{Size of } \boxed{\text{Union}}}$$

“Correct” if IoU 0.5

- It is defined as the ratio of the size of the intersection between the bounding boxes to the union between the boxes.
- It is considered accurate if the IoU ratio comes out bigger than 0.5.

Now, as per the non-max suppression algorithm,

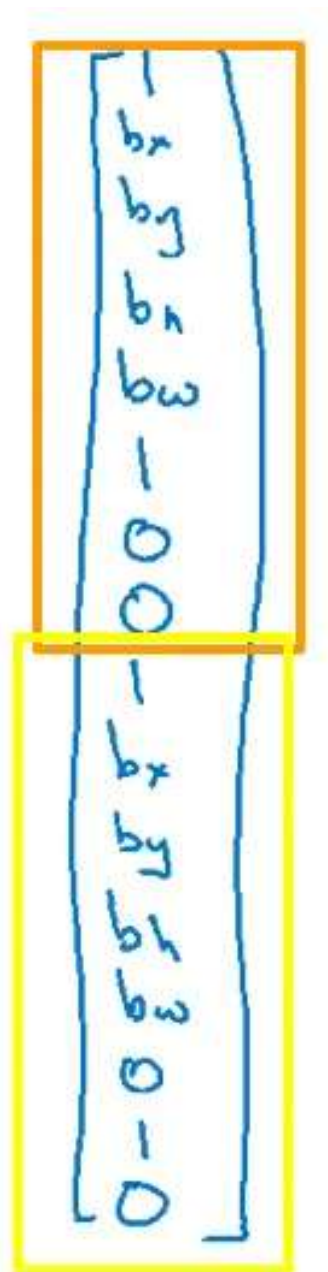
- Pick the box with the largest Output as a prediction i.e. pc value.
- Discard any remaining box using IoU with the box output in the previous step.

— Anchor boxes

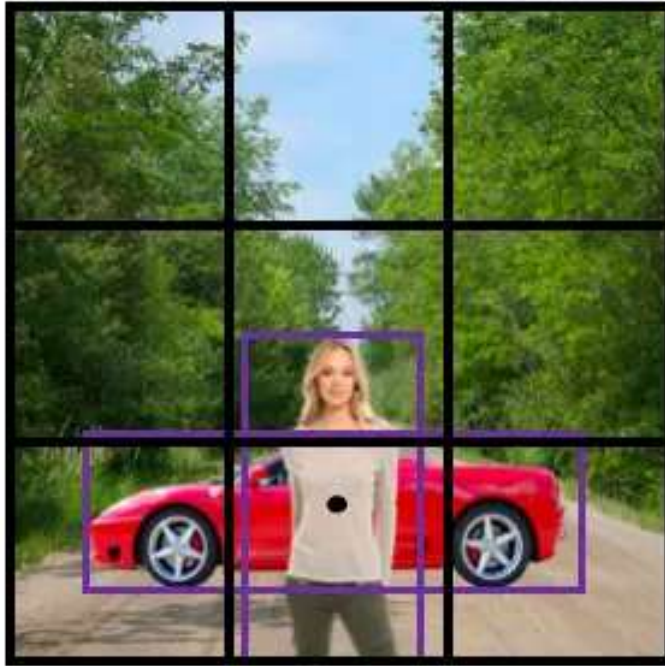
In some cases, more than one object is assigned to the single grid cell as the mid-point of more objects lies in the same grid cell. In order to handle this problem, we use the concept of the anchor boxes.

We define certain shapes of the boxes and try to match various objects with different shapes. Now, it is very rare that more than 2 objects have the same anchor box shape and they are assigned to the same grid cell. If this happens, then it is the drawback of YOLO algorithm.

Due to anchor boxes, our output labels would also change. In the case of two anchor boxes, our output label would be like:



- Previously, each object in the training image is assigned to the grid cell that contains that object's midpoint.
- With two anchor boxes, each object in the training image is assigned to the grid cell that contains the object's midpoint and anchor box for the grid cell with the highest IoU.



Anchor box 1: Anchor box 2:



— Wrap up

So, this is all about object detection and various terms related to it. There is one more algorithm known as SSD(**Single Shot Detector**). SSD is a more robust recommendation. However, if exactness is not too much of matter but you want to go super quick, YOLO will be the best way to move forward.

I would suggest the readers explore SSD and YOLO.

— References:

1. [Object detection by Coursera](#). ****highly recommended****
2. [You Only Look Once: Unified, Real-Time Object Detection](#)

Object Detection

Yolo

Sliding Window Algorithm

Object Localization

Coursera

[Follow](#)

Published in AI n U

55 followers · Last published Aug 28, 2024

This medium publication publishes AI content for AI philomaths, epistemophilias and neophilias. Find or publish your AI thing in your AI way, from any-where, now-here.

[Follow](#)

Written by Rrohan.Arrora

126 followers · 17 following

demystifying the theories, construing the results.

Open in app ↗

[Sign up](#)[Sign in](#)**Medium** Search

